

Statistics in Health Sciences - Assignment 1

G05: Judit Félez Guerrero (17048330), Lúdia Abio Rojo (1705251), Catalina Mascaró Català (1708159), Elia Campos Villaro (1703842)

28th October 2026

1 Introduction

The aim of this assignment is to apply the framework proposed by Basagaña et al. to the data set `copd` in the file `copd redux.RData`. Data `copd` is a subset of 45 variables of the original data analyzed in the cited paper.

2 Analysis

2.1 Description of missingness

Provide the same missingness description than in the original analysis. Specifically, reproduce the following text that is shown in the original paper:

“The subset used here included 342 participants and 85 variables. The range of missing data in the different variables went from 0% to 47.7%, with 68.2% of the variables having less than 5% of the data missing. Only 13.7% of the participants had complete data on all 85 variables. Overall, 5.9% of the information was missing; that is, 5.9% of the total of 29,070 cells ($85 \times 342 = 29,070$) had missing data.”

Instructions:

- Note that, since you are working with a subset of the original data set, red numbers above are not expected to be the same than in your results.
- In your `Rnw` document, the text above must be able to automatically update all red numbers in the case of changing the values of `copd`. I.e., if data change, then all red numbers must also automatically update accordingly in your `pdf` when compiling your `Rnw`. As a hint, you can follow this procedure:
 1. For each red number, create an R object, which will depend on data and gives the desired quantity (i.e., if data change, the value in the object changes accordingly),
 2. To force rounding a quantity to a given decimal precision, you can use `sprintf()` (as seen in Exercise 0). Do not use `round()`,
 3. To get the thousands comma separator as in “29,070”, you can use `prettyNum()`,
 4. Once you have an R object for each red number formatted as desired, use `\Sexpr{}` to insert each of them inline in $\text{\LaTeX}$ (as seen in Exercise 0).

```
> n_obs <- nrow(copd) # Number of participants
> n_vars <- ncol(copd) # Number of variables
> nans <- 100*colMeans(is.na(copd)) # Percentage of missing data by variable
> min_nan <- sprintf("%.2g", min(nans)) # MIRAR # Lower percentage
> max_nan <- sprintf("%.2g", max(nans)) # Upper percentage
>
> # Variables with less than 5% of data missing (percentage)
> less_5 <- sprintf("%.2g", 100*length(nans[nans<5])/n_vars)
>
> # Percentage of complete cases
> comp_cases <- sprintf("%.2g", 100*mean(complete.cases(copd)))
>
> # Percentage of missing data
> miss_total <- sprintf("%.2g", 100*mean(is.na(copd)))
>
> # Total cells observed (variables*observations)
> total_cells = prettyNum(n_obs*n_vars, big.mark = ",", decimal.mark = ".")
```

“The subset used here included 342 participants and 45 variables. The range of missing data in the different variables went from 0% to 32%, with 69% of the variables having less than 5% of the data missing. Only 33% of the participants had complete data on all 45 variables. Overall, 5.4% of the information was missing; that is, 5.4% of the total of 15,390 cells ($45 \times 342 = 15,390$) had missing data.”

2.2 Multiple imputation

Apply multiple imputation to copd data using 100 imputations. Explain in a reasoned manner how you decided to set the values of the arguments `method` and `predictorMatrix` when using the `mice()` function. Show your code.

```
> # Display of default imputation methods for each variable
> ini <- mice(copd, maxit=0)
> ini

## Class: mids
## Number of multiple imputations: 5
## Imputation methods:
##           id index_barthel2v1 quo_calcpstv1 rv_prepv1 incp_fevv1 fevp_postv1
##           ""                  "pmm"         ""         "pmm"         "pmm"
## fevp_postv1 rv_tlcvt1 ctnf_s_mv1 vtdvdbid_ecov1 pp_ecov1 sin11_v1 vas2_v1 ic_prepv1
##           ""                  "pmm"         "pmm"         "pmm"         "pmm"
## sin11_v1 vas2_v1 ic_prepv1 iam_chv1 sin15_v1
## "logreg" "pmm"         "pmm"         "logreg" "logreg"
## cil8_s_mv1 sin7_v1 sin8_v1 ch_v1 vc_prepv1
## "pmm" "logreg" "logreg" "pmm"
## dtdve_ecov1 cccs1_v0 kcals_set_v0 impactsv1 monoA_v1
## "pmm" "logreg" "pmm" "pmm"
## lvmass_ase_ecov1 vtit_catv1 urg_ambv0 po2_v1 ahpv_v1
## "pmm" "logreg" "" "pmm"
## pro_v1 basA_v1 fc_pmv1 tad_v1 col_v1
## "pmm" "pmm" "pmm" "pmm"
## sin5_v1 bc_v1 fvcv_postv1 vas1_v1 sin6_v1
## "logreg" "pmm" "" "pmm" "logreg"
## graudisn_v1 cpcr_mg_l_mediaiv1 neutA_v1 ahpl_v1 activityv1
## "polyreg" "pmm" "pmm" "pmm" "pmm"

## PredictorMatrix:
##           id index_barthel2v1 quo_calcpstv1 rv_prepv1 incp_fevv1 fevp_postv1
## id 0 1 1 1 1 1
## index_barthel2v1 1 0 1 1 1 1
## quo_calcpstv1 1 1 0 1 1 1
## rv_prepv1 1 1 1 0 1 1
## incp_fevv1 1 1 1 1 0 1
## fevp_postv1 1 1 1 1 1 0
##           rv_tlcvt1 ctnf_s_mv1 vtdvdbid_ecov1 pp_ecov1 sin11_v1 vas2_v1 ic_prepv1
## id 1 1 1 1 1 1 1
## index_barthel2v1 1 1 1 1 1 1 1
## quo_calcpstv1 1 1 1 1 1 1 1
## rv_prepv1 1 1 1 1 1 1 1
## incp_fevv1 1 1 1 1 1 1 1
## fevp_postv1 1 1 1 1 1 1 1
##           iam_chv1 sin15_v1 cil8_s_mv1 sin7_v1 sin8_v1 ch_v1 vc_prepv1 dtdve_ecov1
## id 1 1 1 1 1 1 1 1
## index_barthel2v1 1 1 1 1 1 1 1 1
## quo_calcpstv1 1 1 1 1 1 1 1 1
## rv_prepv1 1 1 1 1 1 1 1 1
## incp_fevv1 1 1 1 1 1 1 1 1
## fevp_postv1 1 1 1 1 1 1 1 1
##           cccs1_v0 kcals_set_v0 impactsv1 monoA_v1 lvmass_ase_ecov1 vtit_catv1
## id 1 1 1 1 1 1
## index_barthel2v1 1 1 1 1 1 1
## quo_calcpstv1 1 1 1 1 1 1
## rv_prepv1 1 1 1 1 1 1
## incp_fevv1 1 1 1 1 1 1
## fevp_postv1 1 1 1 1 1 1
##           urg_ambv0 po2_v1 ahpv_v1 pro_v1 basA_v1 fc_pmv1 tad_v1 col_v1 sin5_v1
## id 1 1 1 1 1 1 1 1
## index_barthel2v1 1 1 1 1 1 1 1 1
## quo_calcpstv1 1 1 1 1 1 1 1 1
## rv_prepv1 1 1 1 1 1 1 1 1
## incp_fevv1 1 1 1 1 1 1 1 1
## fevp_postv1 1 1 1 1 1 1 1 1
##           bc_v1 fvcv_postv1 vas1_v1 sin6_v1 graudisn_v1 cpcr_mg_l_mediaiv1 neutA_v1
## id 1 1 1 1 1 1 1
## index_barthel2v1 1 1 1 1 1 1 1
```

```
## quo_calcpstv1      1      1      1      1      1      1      1
## rv_prepv1         1      1      1      1      1      1      1
## incp_fevv1        1      1      1      1      1      1      1
## fevp_postv1       1      1      1      1      1      1      1
##               ahpl_v1 activityv1
## id              1      1
## index_barthel2v1 1      1
## quo_calcpstv1    1      1
## rv_prepv1       1      1
## incp_fevv1      1      1
## fevp_postv1     1      1
```

We can see that the mice package suggest imputing initially with de mean for continuous variables and with a logistic regression for binary ones. Further research on the dataset has shown that the variables have no deterministic relation. So we do not change the imputation methods. We have a variable (`id`) that don't have any role in the imputation process, so we change `predictorMatrix` and `methods` according to that.

```
> # Change in parameters
> meth <- ini$method
> pred <- ini$predictorMatrix
>
> meth["id"] <- "" # do not impute "id"
> pred["id", ] <- 0 # we don't want "id" to be imputed
> pred[, "id"] <- 0 # we don't want "id" to be predictor
```

```
> # Imputation of 100 datasets
> M <- 2 # Number of imputations for each missing value
> imp <- mice(data = copd,
+             m = M,
+             method = meth,
+             predictorMatrix = pred,
+             maxit = 20,
+             printFlag = FALSE,
+             seed = 666)
> # imp$imp (output llarg)
> head(imp$imp)

## $id
## [1] 1 2
## <0 rows> (o 0- extensión row.names)
##
## $index_barthel2v1
##      1  2
## 31 100 100
## 64 100 100
## 187 100 100
## 198 80 100
## 240 90 100
## 241 100 100
## 242 100 100
## 245 100 100
## 302 90 100
##
## $quo_calcpstv1
## [1] 1 2
## <0 rows> (o 0- extensión row.names)
##
## $rv_prepv1
##      1      2
## 31 190.00200 156.37891
## 73 175.15790 140.45828
## 84 179.54126 166.25749
## 85 174.01822 154.18134
## 87 147.11288 142.57919
## 91 266.79022 199.57353
## 96 194.25191 152.39517
```

```
## 99 134.39871 114.54136
## 105 175.47922 154.18134
## 109 304.11612 304.11612
## 110 130.83453 123.91756
## 115 153.99147 173.26074
## 117 233.97670 218.04686
## 121 130.73349 152.96429
## 123 228.64987 94.60959
## 130 114.64743 156.37891
## 139 156.37891 144.07747
## 146 168.70444 128.22917
## 159 87.62526 201.72061
## 172 114.49117 168.70444
## 180 119.85428 135.60764
## 188 137.62262 115.28805
## 189 123.91756 115.28805
## 208 233.97670 128.13847
## 218 147.11288 152.96429
## 277 148.80103 212.59613
## 301 110.02028 177.64473
##
## $incp_fevv1
##          1          2
## 62  4.999995 32.0610771
## 82  7.526881 17.7914085
## 87 11.023622 21.5189819
## 91 -1.408460 -0.4854364
## 126 9.223304 -3.9682503
## 144 6.766911 1.6393427
## 153 11.023622 3.0769293
## 160 0.000000 5.9171538
## 161 2.127658 4.3103509
## 170 -4.186053 -8.0952368
## 179 20.430107 11.0236216
## 186 -4.186053 6.7669106
## 311 -3.968250 4.5454569
## 332 4.237294 4.3902516
##
## $fevp_postv1
## [1] 1 2
## <0 rows> (o 0- extensión row.names)
```

2.3 Integration of multiple imputation in cluster analysis

Reproduce the original analysis in the paper using the `miclust` package to integrate multiple imputation in the cluster analysis. Apply the proposed framework using the same setting than in the original analysis. To do that, follow these steps:

1. Read the help of the `getdata()` function in the `miclust` package. Then, accordingly, create a *list* of *data.frames* and pass it to the `getdata()` function. Give a detailed explanation of the data processing you did to get that *list*. Show your code.
2. Use the `miclust()` function to perform the integration of missing data in cluster analysis using the same settings as in the original analysis. Show your code and explain how and why you set all arguments in the `miclust()` function.